

Table Augmented Generation

Easy relational access

Alessandro Di Girolamo, Rainer Cabral Ilao

Project repository: <https://github.com/aledigirm3/TAG-easy-relational-access>

Keywords: LLM, Compuret Science, Prompt Engineering, RAG, Table, Relational Database

1. Introduction

This project aims to explore a potential solution in the context of relational databases, with a particular focus on table access and query execution. In traditional systems, interaction with relational databases requires knowledge of the SQL language, a skill that general users often lack. In many cases, it would be preferable to provide a more intuitive interface that allows users to formulate queries in natural language and receive coherent responses directly from the database.

The core idea of this work is to introduce an additional layer of abstraction between the user and the database, with the goal of simplifying information access for non-expert users. Some existing solutions translate natural language queries into SQL commands (an approach known as text-to-SQL); however, recent studies show that standard techniques yield correct answers in fewer than 20% of cases, highlighting the need for further improvements in this area.

The proposed project presents a variation of the Retrieval-Augmented Generation (RAG) approach, specifically adapted to the context of querying relational databases through natural language.

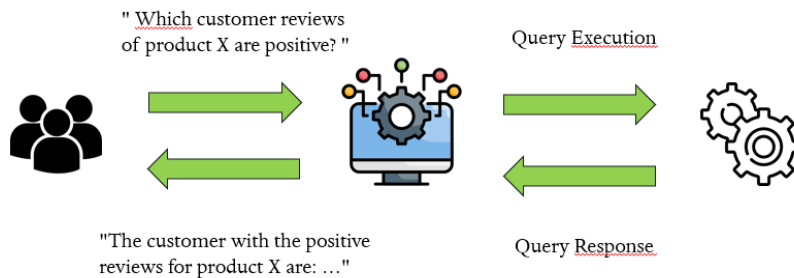


Figure 1: Example of system usage

2. Data preparation

The initial phase of the project focused on preparing the data to be used within the system. This process was structured into several stages, with the objective of transforming the information from relational tables into a format more suitable for processing by language models.

2.1 Pre-Embedding Structure

Relational database tables follow a well-defined structure, optimized for storage and processing through SQL. However, this structure is not well-suited for direct interpretation by large language models (LLMs).

For this reason, before generating the embeddings, it was necessary to transform the table rows into a format more compatible with the capabilities of language models. Specifically, each row (or entry) was converted into a textual representation that makes the content of its attributes explicit.

For example, given a row from a table with multiple attributes, the transformation follows a structure of the form:

```
NameTable, nameAttribute1: attribute1, nameAttribute2, attribute1, ...
```

2.2 Embedding Generation

Once the data had been transformed into the textual format described above, it was possible to proceed with the generation of embeddings—namely, the conversion of strings into high-dimensional vectors suitable for semantic comparison.

To this end, the *all-MiniLM-L6-v2* model, provided by the Python library `sentence-transformers`, was employed. This model is capable of generating dense and effective text representations, well-suited for semantic search and retrieval tasks.

The resulting embeddings were then stored in a vector database, specifically *MilvusDB*. This system supports similarity-based search operations, enabling the retrieval of the most relevant entries based on a query formulated in natural language, which is also converted into an embedding using the same model.

3. Query Execution

Once the data preparation and storage phases in the desired format were completed, the experiment reached its core stage: the execution of natural language queries, with the goal of obtaining coherent and relevant answers based on the knowledge stored in the

database.

The system receives as input a query expressed in natural language, which is transformed into a high-dimensional vector using the same embedding model employed during the preprocessing phase (*all-MiniLM-L6-v2*).

Subsequently, the generated vector is used to query the vector database through a similarity search mechanism. Specifically, *MilvusDB* returns the vectors (and thus the corresponding textual rows) that are most similar to the query vector, based on a suitably selected similarity threshold. In this experiment, the adopted threshold value is set to 0.3.

At this point, the process enters a crucial phase: the construction of the prompt to be provided to the generative model.

For response generation, the *deepseek-r1-distill-llama-70b* model was used, accessed via the Groq platform APIs.

The rows returned by the similarity search are finally parsed and formatted into a coherent structure, which is inserted into the prompt intended for the generative model. This structure is designed to provide the model with sufficient contextual information to formulate an accurate and relevant response to the initial query.

```
Table 'nameTable'  
att1,att2,...  
row1  
row2  
...
```

```
CREATE TABLE (...)
```

The style used to represent the rows in the prompt intentionally resembles a CSV-like format, which is easily interpretable by language models. At the end of the tabular representation, the table schema is also appended the list of attributes and their semantic types. This assists the model in better understanding the data organization and in correctly performing conceptually complex operations, such as implicit JOINS, during response generation.

In addition to the selected rows, the prompt also includes the original natural language query used for semantic filtering. In this way, the model has access both to the informational context (the most relevant data) and to the user’s intent, increasing the likelihood of a precise and context-aware response.

Last but not least, a system instruction (system prompt) was employed to guide the model towards the specific domain of interest. This prompt was refined through an experimental tuning phase aimed at improving its effectiveness in the semantic understanding of tables and in generating accurate responses.

The complete system prompt can be found in the file `src/tag_search.py`.

The following figure illustrates the overall architecture of the system.

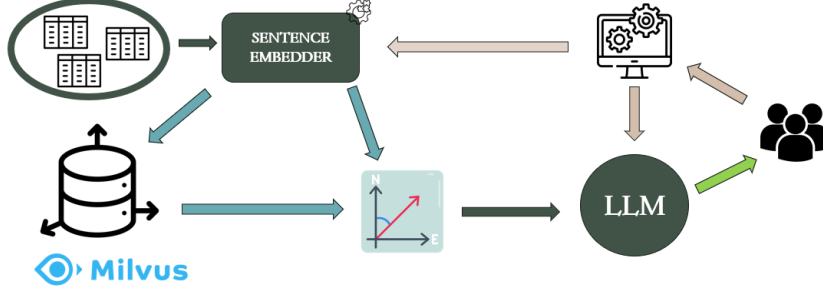


Figure 2: Example of the query execution pipeline

4. Evaluation

This phase is dedicated to assessing the system’s performance, which was tested on three distinct databases to ensure more robust and generalizable results.

The databases used were extracted from the *BIRD dev dataset*¹ and are as follows:

- *student_club*: a database with contained semantic complexity, easily interpretable by a large language model (LLM). It features an average number of tables, although one of them contains a large number of rows.
- *superhero*: similar in structure and size to the previous database, but presenting a slightly greater challenge to the LLM in terms of semantic understanding and a higher number of tables.
- *thrombosis_prediction*: a dataset characterized by high semantic complexity. Although composed of only three tables, these contain complex and numerous pieces of information, representing a significant challenge for the model’s performance in this specialized domain.

Standard metrics of Precision, Recall, and F1-score were adopted to evaluate performance, calculated by comparing the entries obtained from the LLM with those resulting from SQL queries executed directly on the database. Additionally, an extra metric called *uScore* was introduced, based on a human evaluation of the quality of the model-generated responses. It is important to highlight that the evaluation was differentiated according to query complexity, distinguishing between those requiring JOIN operations and those that do not, in order to separately analyze these two scenarios, which are known to differ significantly in computational and semantic complexity (JOIN operations are notably more challenging for the model to handle).

The following table reports the evaluation results considering only queries that do not require JOIN operations.

¹<https://bird-bench.github.io/>

	Precision	Recall	F1-score	uScore
student_club 1	0.85	0.84	0.845	3.7
superhero	0.75	0.5	0.6	3.27
thrombosis_prediction	0.778	0.46	0.58	2.7
AVG	0.82	0.67	0.74	3.28

Table 1: TAG Performance without JOIN operations

This table, instead, reports the system’s performance on queries involving JOIN operations.

	Precision	Recall	F1-score	uScore
student_club 1	0.43	0.46	0.44	2.6
superhero	0.5	0.28	0.36	2.57
thrombosis_prediction	0.5	0.25	0.33	2.5
AVG	0.44	0.42	0.43	2.57

Table 2: TAG Performance with JOIN operations

As expected, the results significantly declined when the system was required to perform the JOIN operation (implicitly). Furthermore, the dataset on which the system performed best in the absence of JOINS was *student_club*, being the semantically simplest among the three.

5. Conclusions

This project explored the application of a system based on Retrieval-Augmented Generation (RAG) to facilitate querying relational databases through natural language, overcoming the traditional barrier posed by the need to know SQL.

Through careful data preparation, semantic embedding generation, and the use of a vector database for similarity search, it was possible to build a workflow capable of providing relevant answers to natural language queries. Integration with an advanced generative model, specialized through system prompts and tuning, further improved the quality of the responses.

Tests conducted on datasets of varying complexity demonstrated that, although the system shows promise in handling simple to medium-complexity queries, performance tends to decrease when faced with complex queries requiring JOIN operations, confirming the intrinsic challenge of effectively modeling relational logic using LLM-based methods.

In conclusion, this work demonstrates how the RAG approach can represent a viable direction for making relational database querying more accessible and intuitive, while

highlighting the need for further research and optimizations to improve the handling of more complex queries.

Future developments may include the integration of specialized models for understanding relationships between tables, as well as the adoption of hybrid techniques combining symbolic intelligence with machine learning to enhance the system's accuracy and robustness.